

AI Remote Control System Architecture

Overview

The AI Remote Control System is a full-stack platform that allows a mobile application to remotely control a desktop computer, execute commands, stream screens, manage files, and leverage AI agents for autonomous task execution.

The architecture consists of five major layers:

1. Mobile Application (React Native)
 2. Transport Layer
 3. Backend Services (FastAPI)
 4. AI Engine
 5. Desktop Agent
 6. Infrastructure & DevOps
-

1. Mobile Application Layer

Technology Stack

- React Native
- TypeScript
- Zustand / Redux
- WebSocket Client
- Secure Storage

The mobile application serves as the primary user interface and command center.

Navigation Architecture

Navigation Components

- AppNavigator
- AuthNavigator
- MainTabNavigator
- Deep Linking Configuration

Responsibilities

- Route management
- Authentication flow

- Device navigation
 - Screen transitions
 - Deep-link handling
-

Application Screens

Authentication

Features:

- Login
 - Registration
 - Device Pairing
 - QR Pairing
-

Dashboard

Features:

- Device status
 - System health metrics
 - Recent activity
 - Quick actions
-

Remote Control

Features:

- Live desktop stream
 - Mouse control
 - Keyboard control
 - Window management
-

Terminal

Features:

- Remote shell access
 - Command execution
 - Real-time logs
 - Script execution
-

Files

Features:

- Browse files
 - Upload files
 - Download files
 - Search files
-

AI Assistant

Features:

- Chat interface
 - Voice commands
 - AI workflows
 - Autonomous task execution
-

Applications

Features:

- Installed application list
 - Launch applications
 - Kill processes
 - Window switching
-

Monitoring

Features:

- CPU monitoring
 - RAM monitoring
 - Network statistics
 - GPU monitoring
-

Connection

Features:

- Connected devices
- Device history

- Pairing management
-

Settings

Features:

- Profile management
 - Security controls
 - Permissions
 - Theme management
-

State Management

Stores

authStore

Responsible for:

- User session
 - Authentication status
 - JWT tokens
-

deviceStore

Responsible for:

- Device registry
 - Device selection
 - Device state
-

connectionStore

Responsible for:

- Connectivity status
 - Reconnection state
 - Heartbeat tracking
-

terminalStore

Responsible for:

- Terminal logs
 - Command history
 - Active sessions
-

fileStore

Responsible for:

- File metadata
 - Downloads
 - Uploads
-

streamStore

Responsible for:

- Video streams
 - FPS metrics
 - Quality management
-

aiStore

Responsible for:

- Chat history
 - AI workflows
 - Agent state
-

Mobile Services

API Client

Handles:

- Authentication APIs
- Device APIs
- File APIs
- AI APIs

WebSocket Client

Handles:

- Terminal streams
- Live events
- Notifications
- Screen streaming

Secure Storage

Stores:

- JWT tokens
- Refresh tokens
- Cached user data

Stream Service

Handles:

- Video playback
- FPS monitoring
- Quality adjustments

Custom Hooks

useWebSocket

WebSocket management.

useAuth

Authentication logic.

useDeviceConnection

Device connectivity management.

useTerminal

Terminal operations.

useFileTransfer

File upload/download logic.

useScreenStream

Live screen stream management.

2. Transport Layer

The transport layer connects the mobile app, backend, AI engine, and desktop agent.

HTTPS REST API

Used for:

- Authentication
- CRUD operations
- File transfers
- Device management

Security:

- JWT Authentication
 - TLS Encryption
-

WebSockets

Used for:

- Terminal output
- Device events
- Notifications
- AI responses

Features:

- Real-time communication
 - Low latency
 - Persistent connection
-

WebRTC (Optional)

Used for:

- Screen streaming
 - Audio streaming
 - Remote desktop interaction
-

3. Backend Layer

Technology

- FastAPI
- Python
- SQLAlchemy
- Redis
- PostgreSQL

The backend acts as the central orchestration hub.

REST API Layer

auth.py

Responsibilities:

- Login
 - Registration
 - Token refresh
 - Logout
-

devices.py

Responsibilities:

- Device registration
 - Pairing
 - Device CRUD operations
-

commands.py

Responsibilities:

- Execute commands
 - Command history
 - Command auditing
-

files.py

Responsibilities:

- Upload files
 - Download files
 - File metadata
-

ai.py

Responsibilities:

- AI chat
 - AI workflows
 - Task execution
-

streaming.py

Responsibilities:

- Screen stream management
 - Monitoring streams
-

WebSocket Layer

connection_manager

Handles:

- Rooms
 - Broadcasting
 - Device sessions
-

terminal_handler

Handles:

- PTY sessions
 - Terminal output
-

stream_handler

Handles:

- Video frames
 - Stream quality
-

heartbeat_handler

Handles:

- Ping/Pong
 - Connection recovery
-

Business Services

auth_service

Responsibilities:

- Password hashing
- JWT generation

Tools:

- bcrypt
 - JWT
-

device_service

Responsibilities:

- Device registration

- Token generation
-

command_service

Responsibilities:

- Route commands
 - Audit execution
-

file_service

Responsibilities:

- Local storage
 - Cloud storage
-

monitoring_service

Responsibilities:

- Push metrics
 - Health monitoring
-

Middleware Layer

Includes:

- JWT Authentication
 - Rate Limiting
 - Structured Logging
 - Exception Handling
-

Data Layer

PostgreSQL

Tables:

- users
 - devices
 - sessions
-

Redis

Stores:

- Sessions
 - Cache
 - WebSocket state
-

SQLAlchemy + Alembic

Responsibilities:

- Models
 - Database migrations
 - Async database access
-

4. AI Engine Layer

Technology:

- LangChain
- Custom Agent Framework
- OpenAI
- Claude
- Ollama

The AI Engine transforms natural language into executable workflows.

Intent Parser

Responsibilities:

- Natural language understanding
- Entity extraction

Example:

Input:

Open VS Code and run app.py

Output:

```
{  
  "app": "vscode",  
  "action": "run",  
  "target": "app.py"  
}
```

Task Planner

Creates:

- Execution DAGs
- Task dependency graphs
- Sequential workflows

Orchestrator

Responsibilities:

- Execute plans
 - Retry failures
 - Monitor progress
-

Tool Executor

Provides access to:

- Shell
 - Mouse
 - Keyboard
 - File system
 - Applications
-

Memory Manager

Stores:

- Session context
- User preferences
- Historical tasks

Vector Storage:

- ChromaDB
 - Pinecone
 - FAISS
-

LLM Provider Layer

Supported Models:

- GPT-4
- Claude
- Ollama Local Models

Switchable at runtime.

Available Tools

Command Execution

- run_command

Desktop Control

- move_mouse

- click
- type_text

Application Management

- open_app

File Operations

- read_file
- write_file

Monitoring

- get_metrics

Visual Tools

- take_screenshot
-

5. Desktop Agent Layer

Technology:

- Python

Runs on the user's computer.

Connection Subsystem

websocket_client

Maintains backend communication.

connection_manager

Session management.

heartbeat

Health reporting.

reconnection_handler

Auto recovery.

device_info

Hardware and OS reporting.

Automation Layer

Desktop Automation

Components:

- mouse_controller
- keyboard_controller
- app_launcher
- window_manager
- hotkey_manager

Capabilities:

- Mouse movement
 - Keyboard typing
 - Application launching
 - Window control
-

Terminal Subsystem

Components:

- command_executor
- shell_handler
- process_manager
- output_streamer

Capabilities:

- Run commands
 - Execute scripts
 - Stream outputs
-

Streaming Layer

Screen Streaming

Components:

- screen_capturer
- frame_encoder
- quality_optimizer
- multi_monitor

Capabilities:

- Live screen sharing
 - Multiple monitors
 - Adaptive quality
-

File System Layer

File Manager

Components:

- file_manager
- uploader
- downloader
- file_watcher
- directory_browser

Capabilities:

- Upload files
 - Download files
 - Browse directories
 - Watch changes
-

System Monitoring Layer

Resource Tracking

Tracks:

- CPU
 - RAM
 - Disk
 - GPU
-

Process Monitoring

Tracks:

- Running applications
 - Background services
-

Network Monitoring

Tracks:

- Bandwidth
 - Latency
 - Connectivity
-

Power Management

Tracks:

- Battery
 - Sleep state
 - Shutdown events
-

Security Layer

Authenticator

Validates:

- Device tokens
 - Sessions
-

Permission Checker

Controls:

- Allowed actions
 - Restricted operations
-

Command Sanitizer

Prevents:

- Dangerous execution
 - Malicious commands
-

Audit Logger

Logs:

- Commands
 - User actions
 - Device events
-

6. Infrastructure & DevOps

Supports deployment and scaling.

Docker

Containers:

- Backend
 - Desktop Agent
 - NGINX
-

Kubernetes

Resources:

- Deployments
 - Services
 - Ingress
 - ConfigMaps
-

Monitoring

Tools:

- Prometheus
 - Grafana
-

CI/CD

Platform:

- GitHub Actions

Pipelines:

- Backend
 - Mobile App
 - Desktop Agent
-

Desktop Agent Installers

Windows

windows_installer.ps1

macOS

mac_installer.sh

Linux

linux_installer.sh

Shared Setup

permissions_setup.py

Complete Request Flow

1. User enters command on mobile app.
 2. Command sent through REST/WebSocket.
 3. Backend authenticates request.
 4. AI Engine interprets intent.
 5. Task Planner creates workflow.
 6. Orchestrator selects tools.
 7. Backend forwards task to Desktop Agent.
 8. Desktop Agent executes task.
 9. Output streamed back.
 10. AI verifies success.
 11. Final result displayed on mobile device.
-

Final Architecture Summary

The system consists of:

- React Native Mobile App

- FastAPI Backend
- LangChain AI Engine
- Python Desktop Agent
- PostgreSQL Database
- Redis Cache
- WebSocket Communication
- Optional WebRTC Streaming
- Docker + Kubernetes Deployment

This architecture supports:

- Remote desktop control
- AI automation
- Terminal execution
- Screen streaming
- File management
- Real-time monitoring
- Autonomous AI workflows
- Enterprise-scale deployment